VNA screening

Overview

The VNA screening scripts in the procedures folder take full VNA sweeps of a device with the CRS, then find and fit its resonances. Each script is a short file with a settings block at the top. Copy the script, edit the settings for your setup, and run it. Command-line options override the settings for one run.

Script	When to run it	What it does	Saves
`crs_init.py`	After a power cycle, or to change board-wide settings	Configures the board	Nothing
`vna_quick_check.py`	Any time	Takes quick VNA sweeps of one device in an interactive window	Optionally, a CSV of f, I, Q
`01_vna_sweep.py`	Once per device, or after a change to the setup	Takes a full VNA sweep	`vna{k}`
`02_find_resonances.py`	After each VNA sweep	Finds, checks, and fits the resonances	`full_sweep_fit`, `resonators{k}`
`vna_report.py`	After `02_find_resonances.py`	Writes a plot, a text summary, and a table of the results	Files in a folder

Run the scripts in citkid's environment, for example:

```
uv run python procedures/01_vna_sweep.py --device deviceA --modules 1
```

The CNN resonance finder used by `02_find_resonances.py` needs PyTorch, which is installed once with `uv sync --extra cnn`.

The device store

Each device has one zarr store per cooldown, `<DATA_DIR>/<COOLDOWN>/<DEVICE>.zarr`, which holds all of the data measured and analyzed for that device:

```
<device>.zarr/
  vna{k}/            full VNA sweep (raw), with the resonance finding and
                     fitting results
    crs_config/        CRS settings used for the sweep
    fres_auto, res_finder_auto/, fres_manual   resonance frequencies found
    full_sweep_fit/    fits of every resonance (analysis)
  resonators{k}/     resonator list from vna{k}, used for tuning
  tune{k}/           tunings (see the tuning and timestreams document)
```

Indices are not zero-padded (`vna0`, `vna1`, ...). Raw data is never overwritten: repeating a measurement saves it under the next index. The store records which CRS it was measured with (serial number, firmware version, and rfmux version). A different board or firmware version needs a new store.

Notes with the data

Every script that saves data has a `NOTES` setting (or `--notes`). Its text, which can span several lines, is saved with the data in the attribute `notes`. Use it to record, for example, the filters and attenuators on the DAC and ADC lines.

Measuring several devices at once

`crs_init.py` takes global control of the board to configure it. The other scripts only attach to their own modules (`--modules`), so several of them can run at the same time on different devices.

Timestreams are the exception. Streaming uses the board's full network bandwidth, so `04_stream.py` needs exclusive use of the board for as long as its window is open. During that time, any other script that tries to use the board, including the scripts in this document, stops immediately with an error that names the running `04_stream.py` session. Run the script again after that window is closed. The lock is a file on the DAQ computer (`~/.citkid/locks/crs<serial>.lock`), so it only applies to scripts run on that computer. If a session crashes, its lock is ignored.

crs_init.py: configure the board

Run this script before any of the others, and again whenever the board loses its configuration (for example, after a power cycle). It is the only script that changes board-wide settings:

- the clock source;
- the analog bank (modules 1-4, or 5-8);
- the DAC full scale (always 7 dBm);
- the module bandwidth (500 MHz, or 600 MHz with `--extended-bw`);
- the decimation stage used for sweeps (stage 6).

The script exits when it is done, and the board keeps these settings. Running it while other scripts are measuring resets the board in the middle of their measurements.

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
CLOCK_SOURCE = 'VCXO'       # 'VCXO' (internal) or 'SMA' (external 10 MHz)
ANALOG_BANK_HIGH = False    # False: modules 1-4; True: modules 5-8
EXTENDED_BW = False         # True: 600 MHz module bandwidth (more loss
                            # near the band edges)
```

Command line

```
usage: crs_init.py [-h] [--serial SERIAL] [--iface IFACE] [--clock {VCXO,SMA}]
                   [--bank-high] [--extended-bw]

Configure the CRS board.

options:
  -h, --help          show this help message and exit
  --serial SERIAL
  --iface IFACE
  --clock {VCXO,SMA}
  --bank-high
  --extended-bw
```

vna_quick_check.py: quick look at a device

This script opens a window for checking the wiring and the tone power, and for checking that the resonances are there before the full sweep. Nothing is saved to the store. The last sweep can be saved as a CSV file with columns `f` (Hz), `i`, and `q` (Save data).

The script does not configure the board, so run `crs_init.py` first. It connects to the board given by the serial number and network interface settings, and attaches to its modules without changing the board-wide settings, so it can run alongside the other scripts. The modules must be in the analog bank chosen by `crs_init.py` (1-4, or 5-8 with `--bank-high`). Modules in the other bank are listed in the status line and not used.

The window has these controls:

- **Modules**: the modules connected to the device. Use several if a splitter feeds the device to more than one module. Type them (for example `1 2` or `5 6`) and press Connect to change them.
- **Nyquist zone**: zone 1 (100-2500 MHz), zone 2 (2500-5000 MHz), or both. Sweeps skip 10 MHz on each side of 2500 MHz, because the point nearest the Nyquist frequency is unreliable. The range can also be set by hand (From, To).
- **Resolution**: the frequency step is set by the number of points per linewidth at the expected median Qr, so df/f = 1 / (median Qr x points per linewidth). The points are spaced geometrically, so every resonance with the same Qr gets the same number of points per linewidth.
- **Plan**: shows the number of points, bands, and passes, the points per channel, and the time the sweep will take.
- **Sweep** (Ctrl+R): takes the sweep with a progress bar, and plots |S21| and phase with the band edges marked. If "Count resonances after sweeping" is checked, the resonances are counted with a matched filter (a rough count) and marked on the plot. **Count now** counts the resonances of the last sweep without taking new data.

With `--no-gui` (for example, over ssh without a display), the script takes one sweep with the settings, prints the resonance count, and saves the sweep as a CSV if `--csv` is given.

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
MODULES = [1]               # modules connected to this device
NYQUIST = '1'               # '1' (100-2500 MHz), '2' (2500-5000), 'both'
AMPLITUDE = -50.            # tone power, dBm
QR_MEDIAN = 2e4             # expected median Qr
PTS_PER_LW = 5.             # points per linewidth at QR_MEDIAN (df/f = 1e-5)
NSAMPS = 10                 # samples averaged per point
```

Command line

```
usage: vna_quick_check.py [-h] [--serial SERIAL] [--iface IFACE]
                          [--modules MODULES [MODULES ...]]
                          [--nyquist {1,2,both}] [--fmin FMIN] [--fmax FMAX]
                          [--amplitude AMPLITUDE] [--qr-median QR_MEDIAN]
                          [--pts-per-lw PTS_PER_LW] [--nsamps NSAMPS]
                          [--no-count] [--no-gui] [--csv CSV]

Take quick VNA sweeps of a device.

options:
  -h, --help            show this help message and exit
  --serial SERIAL
  --iface IFACE
  --modules MODULES [MODULES ...]
  --nyquist {1,2,both}
  --fmin FMIN           override the zone's lower edge, Hz
```

```
  --fmax FMAX           override the zone's upper edge, Hz
  --amplitude AMPLITUDE
  --qr-median QR_MEDIAN
  --pts-per-lw PTS_PER_LW
  --nsamps NSAMPS
  --no-count            don't count resonances
  --no-gui              sweep once without the window
  --csv CSV             with --no-gui: save the sweep (f, i, q) here
```

01_vna_sweep.py: full VNA sweep

This script takes a full VNA sweep of one device and saves it as the next `vna{k}` in the device store. If the store doesn't exist yet, it is created, with the `SETUP` setting saved as a description of the setup (fridge channel, attenuators, filters, ...).

- The sweep covers the chosen Nyquist zone or zones (`NYQUIST`). The points are spaced geometrically, with df/f = 1 / (`QR_MEDIAN` x `PTS_PER_LW`). The default is 25 points per linewidth at Qr = 2e4 (df/f = 2e-6), which works best for the resonance finders and the fitter: 10-80 points per linewidth for a typical resonance, and at least 5 for the narrowest.
- Every channel sweeps downward in frequency. The range is divided into bands, each no wider than the module bandwidth and with at most 1024 channels. If there are more bands than modules, the bands are measured in several passes. The number of points per channel is chosen to make the sweep as fast as possible (`--npoints` overrides it).
- With several modules (through a splitter), the bands are divided among the modules and measured in parallel.
- The script saves `f` and `z` (combined into one sweep in ascending frequency), the band arrays (edges, NCO frequencies, modules, passes, Nyquist zones, channels, and start times), the sweep settings, and the CRS settings (`crs_config`).

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
DATA_DIR = '~/data'         # stores go in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
# Free-form description saved with a new store (fridge channel,
# attenuators, filters, ...)
SETUP = dict(fridge_ch = None, dac_attenuation = None, dac_filters = None,
             adc_filters = None)
MODULES = [1]               # modules connected to this device
NYQUIST = '1'               # '1' (100-2500 MHz), '2' (2500-5000), 'both'
AMPLITUDE = -55.            # tone power, dBm
QR_MEDIAN = 2e4             # expected median Qr
# Points per linewidth at QR_MEDIAN (25 gives df/f = 2e-6); more than 10
# is best for fitting
PTS_PER_LW = 25.
NSAMPS = 100                # samples averaged per point

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: 01_vna_sweep.py [-h] [--serial SERIAL] [--iface IFACE]
```

```
                      [--data-dir DATA_DIR] [--cooldown COOLDOWN]
                      [--device DEVICE] [--modules MODULES [MODULES ...]]
                      [--nyquist {1,2,both}] [--fmin FMIN] [--fmax FMAX]
                      [--amplitude AMPLITUDE] [--qr-median QR_MEDIAN]
                      [--pts-per-lw PTS_PER_LW] [--npoints NPOINTS]
                      [--nsamps NSAMPS] [--notes NOTES]

Take a full VNA sweep.

options:
  -h, --help            show this help message and exit
  --serial SERIAL
  --iface IFACE
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --modules MODULES [MODULES ...]
  --nyquist {1,2,both}
  --fmin FMIN           override the zone's lower edge, Hz
  --fmax FMAX           override the zone's upper edge, Hz
  --amplitude AMPLITUDE
  --qr-median QR_MEDIAN
  --pts-per-lw PTS_PER_LW
  --npoints NPOINTS     points per channel (default: fastest)
  --nsamps NSAMPS
  --notes NOTES         notes saved with the data (quote multi-line text)
```

02_find_resonances.py: find and fit the resonances

This script analyzes a saved VNA sweep (by default the latest; choose another with `--sweep k`). It does not need the CRS. It runs these steps:

1. **find**: The CNN finder (default) or the interactive automatic finder (`--finder auto`) finds the resonances and saves their frequencies in `fres_auto`, with the finder's settings and results in `res_finder_auto/`.
2. **manual**: Check and edit the list of resonances in the manual finder window, which saves `fres_manual`. With `--no-manual`, the finder's list is used as it is (useful for simulated data).
3. **fit**: Every resonance is fit (overlapping resonances are fit together), the gain of the sweep is fit, and a summary of the sweep is printed. The fits are saved in `full_sweep_fit`, along with each resonance's flags and which of its parameters are reliable.
4. **resonators**: Save the resonator list used for tuning, `resonators{k}`, in ascending frequency. It holds the fitted resonance frequencies (or the finder's frequencies where the fit failed), Qr where it is reliable (used for linewidths), the target sweep span factors Qr / `SPAN_LW` (so that target sweeps span `SPAN_LW` linewidths), and the dip depths, which `03_tune.py` uses to choose between close resonators. The depths are measured directly on the gain-removed sweep, as 1 - min |S21| within half a linewidth of each resonance (and never past halfway to its nearest neighbor), so they don't depend on the fit: the fitted Qr/Qc is unreliable for exactly the overlapping resonances being compared.

Steps that are already done are skipped, so the script picks up where it stopped. `--redo <step>` repeats that step and the steps after it. A resonator list that is used by a tuning is never replaced; take a new VNA sweep to make a new list.

Which fitted parameters are reliable

The independent fit parameters are fr, Qi, Qc, and phi. Qr and amp are computed from Qi and Qc and are as reliable as Qc.

Fit result	fr	Qi	Qc, phi (and Qr, amp)
Good fit	yes	yes	yes
Overlap: a neighboring resonance affects the fitted pole but not the zero	yes	yes	no
Poor fit (large residuals), distorted, or beyond bifurcation	yes	no	no
Failed fit	no	no	no

These results are saved as `valid_fr`, `valid_Qi`, `valid_Qc`, and `valid_phi` in `full_sweep_fit`. Where the fit failed, the resonance frequency in `resonators{k}` and in the report is the frequency that the resonance finder passed to the fitter (`fres_input`), not a fitted value, and `valid_fr` is False.

Settings

```
DATA_DIR = '~/data'          # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'        # cooldown name
DEVICE = 'device'            # device name
FINDER = 'cnn'               # 'cnn' (automatic) or 'auto' (interactive)
SPAN_LW = 5.                 # target sweep span, linewidths (qres = Qr/SPAN_LW)

STEPS = ('find', 'manual', 'fit', 'resonators')

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: 02_find_resonances.py [-h] [--data-dir DATA_DIR] [--cooldown COOLDOWN]
                             [--device DEVICE] [--store STORE] [--sweep SWEEP]
                             [--finder {cnn,auto}]
                             [--redo {find,manual,fit,resonators}]
                             [--no-manual] [--span-lw SPAN_LW] [--notes NOTES]

Find and fit resonances.

options:
  -h, --help            show this help message and exit
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE         store path (instead of data dir/cooldown/device)
  --sweep SWEEP         VNA sweep index (default: the latest)
  --finder {cnn,auto}
  --redo {find,manual,fit,resonators}
                        repeat this step and the ones after it
  --no-manual           skip the manual check and use the finder's list as it
                        is (for simulations and quick looks)
  --span-lw SPAN_LW
  --notes NOTES         notes saved with the data (quote multi-line text)
```

vna_report.py: results in plain files

This script writes the reduced results of a VNA sweep in human-readable form: a plot, a text summary, and a table. Run it after `02_find_resonances.py`. It writes these files to a folder:

- `<device>_vna<k>.png`: the sweep with the gain removed and the resonances marked (open circles where the fit is unreliable), and plots of Qi, Qc, and phi vs frequency for the reliable fits.
- `<device>_vna<k>_info.txt`: the sweep settings (power, averaging, resolution, frequency range, modules, Nyquist zones, and bands), how the resonances were found and fit, the number of resonances, how they are distributed in frequency (range, gaps, and banks), typical quality factors, data-quality notes, and the user's notes.
- `<device>_vna<k>_resonators.csv`: one row per resonator, with `res_idx` matching `resonators{k}`. The columns are fr, Qi, Qc, Qr, amp, and phi with their errors (NaN where unreliable), the flags, the fit status, and the `valid_*` columns. `fr` is never NaN: where the fit failed, it is the finder's frequency, with `valid_fr` False and `fr_err` NaN.

Settings

```
DATA_DIR = '~/data'         # stores are in DATA_DIR/COOLDOWN/DEVICE.zarr
COOLDOWN = 'cooldown'       # cooldown name
DEVICE = 'device'           # device name
OUT_DIR = '~/data/reports'  # where the files go
```

Command line

```
usage: vna_report.py [-h] [--data-dir DATA_DIR] [--cooldown COOLDOWN]
                     [--device DEVICE] [--store STORE] [--sweep SWEEP]
                     [--out OUT]

Write the results of a VNA sweep to plain files.

options:
  -h, --help           show this help message and exit
  --data-dir DATA_DIR
  --cooldown COOLDOWN
  --device DEVICE
  --store STORE        store path (instead of data dir/cooldown/device; may be
                       a zip)
  --sweep SWEEP        VNA sweep index (default: the latest fitted)
  --out OUT            output directory
```

Trying it on simulated data

`simulate_store.py` makes a store with a VNA sweep of a simulated device (banks of resonators, close pairs, nonlinearity, a realistic gain, and noise). The sweep is taken with the same procedure as a real sweep, and the device's true parameters are saved with it. `02_find_resonances.py` and `vna_report.py` then run on it the same way as on real data, and `citkid.vna.simulate.compare_to_truth` compares the fits with the true parameters.

Settings

```
STORE = '~/data/sim/simdevice.zarr'   # new store path
SEED = 0                    # device, gain, and noise seed
NYQUIST = '1'               # '1' (100-2500 MHz), '2' (2500-5000), 'both'
AMPLITUDE = -55.            # tone power, dBm
QR_MEDIAN = 2e4             # expected median Qr (sets the step)
PTS_PER_LW = 25.            # points per linewidth at QR_MEDIAN
NSAMPS = 100                # samples averaged per point
SNR_DB = 40.                # signal-to-noise ratio per quadrature at
```

```
                        # -55 dBm with 100 samples averaged

# Notes saved with the data this script makes (attribute 'notes'), e.g.
# the filters and attenuators on the DAC and ADC lines. Multi-line is fine.
NOTES = """
"""
```

Command line

```
usage: simulate_store.py [-h] [--store STORE] [--seed SEED]
                         [--nyquist {1,2,both}] [--amplitude AMPLITUDE]
                         [--qr-median QR_MEDIAN] [--pts-per-lw PTS_PER_LW]
                         [--nsamps NSAMPS] [--snr-db SNR_DB] [--notes NOTES]

Make a store with a simulated device.

options:
  -h, --help            show this help message and exit
  --store STORE
  --seed SEED
  --nyquist {1,2,both}
  --amplitude AMPLITUDE
  --qr-median QR_MEDIAN
  --pts-per-lw PTS_PER_LW
  --nsamps NSAMPS
  --snr-db SNR_DB
  --notes NOTES         notes saved with the data (quote multi-line text)
```